

Natural Language Processing

Core Concepts Handout

SUMMER 2026 — BSIT, 3RD SEMESTER

Foundations • Text Processing • Language Models • Representations • Retrieval •
Sequence Models • Transformers • Speech • Ethics

Contents

1	Introduction to Natural Language Processing	2
1.1	Why Language Is Hard for Machines	2
1.2	A Brief History of NLP	2
1.3	Levels (Layers) of Language Analysis	2
2	The NLP Pipeline and Text Preprocessing	3
2.1	Core Preprocessing Steps	3
3	Morphological and Lexical Analysis	4
3.1	Types of Morphemes	4
3.2	Lexical Analysis and Vocabulary	4
4	Regular Expressions and Finite-State Methods	4
4.1	Common Regex Building Blocks	4
4.2	Finite-State Automata (FSA) for Morphology	5
5	Syntax and Parsing	5
5.1	Part-of-Speech (POS) Tagging	5
5.2	Grammar and Parse Trees	5
5.3	Constituency vs. Dependency Parsing	6
6	N-gram Language Models	6
6.1	The Chain Rule and the Markov Assumption	6
6.2	Smoothing	6
6.3	Perplexity	7
7	Text Representation Techniques	7
7.1	One-Hot Encoding and Bag-of-Words	7
7.2	TF-IDF	7
7.3	Word Embeddings	7
7.4	Cosine Similarity	8

8	Information Retrieval Basics	8
8.1	The Vector Space Model for Search	8
8.2	Inverted Index	8
8.3	Precision and Recall in Retrieval	8
9	Semantic Analysis	9
9.1	Word Sense Disambiguation (WSD)	9
9.2	Named Entity Recognition (NER)	9
9.3	Semantic Role Labeling (SRL)	9
9.4	Coreference Resolution	9
10	Sequence Models for NLP	9
10.1	Hidden Markov Models (HMMs) for POS Tagging	10
10.2	RNNs, LSTMs, and GRUs	10
10.3	Sequence-to-Sequence (Seq2Seq) Models	10
11	Attention and Transformer Architecture	10
11.1	Self-Attention Mechanism	10
11.2	Transformer Encoder-Decoder	11
11.3	BERT, GPT, and Large Language Models	11
12	Core NLP Tasks and Applications	11
12.1	Common NLP Tasks	11
12.2	Applications of NLP	12
13	Speech Processing Basics	12
13.1	Automatic Speech Recognition (ASR)	12
13.2	Text-to-Speech (TTS)	12
14	Evaluation Metrics in NLP	13
14.1	Classification Metrics (Recap)	13
14.2	BLEU Score (Machine Translation)	13
14.3	ROUGE Score (Summarization)	13
14.4	Perplexity (Recap)	13
15	Ethical and Responsible NLP	13
16	Core Mathematical Foundations for NLP	14
16.1	Probability Recap	14
16.2	Why Use Log Probabilities?	14
16.3	Vectors and Similarity Recap	14
17	Exam-Oriented Worked Examples	14
18	Quick Revision Glossary	16

19 Question Bank Blueprint (80 Marks Support)	17
19.1 Important Theory Questions	18
19.2 Important Numerical/Analytical Questions	18
19.3 Sample MCQs with Answer Key	18
20 Summary	19

1 Introduction to Natural Language Processing

📖 Natural Language Processing

Natural Language Processing (NLP) is the branch of Artificial Intelligence concerned with enabling computers to **read**, **understand**, **interpret**, and **generate** human language in a useful way, bridging human communication and machine computation.

💡 Why NLP Matters

NLP powers search engines, machine translation, virtual assistants (Siri, Alexa), spam filters, chatbots, sentiment analysis for businesses, and modern large language models such as ChatGPT. For BSIT students, NLP skills are essential for building intelligent text- and speech-based applications.

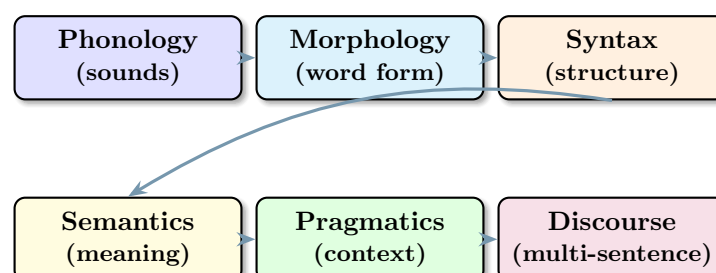
1.1 Why Language Is Hard for Machines

- **Lexical Ambiguity:** A word has multiple meanings, e.g., “bank” (riverbank vs. financial bank).
- **Syntactic Ambiguity:** A sentence has more than one valid grammatical structure, e.g., “I saw the man with the telescope.”
- **Semantic Ambiguity:** A sentence can have multiple possible meanings even with one structure.
- **Referential Ambiguity:** Pronouns can refer to more than one entity, e.g., “Ali told John he passed the exam” — who passed?
- **Context Dependence:** Meaning often depends on world knowledge, tone, and prior conversation.

1.2 A Brief History of NLP

- **1950s:** Alan Turing’s ideas and early rule-based machine translation experiments.
- **1960s–1980s:** Rule-based and grammar-based systems (e.g., the ELIZA chatbot, 1966).
- **1990s–2000s:** Statistical NLP — probability and machine learning replace hand-written rules (n-gram models, Hidden Markov Models).
- **2010s:** Word embeddings (Word2Vec, GloVe) and deep learning (RNNs, LSTMs) transform NLP.
- **2017–present:** The Transformer architecture enables BERT, GPT, and modern Large Language Models (LLMs).

1.3 Levels (Layers) of Language Analysis

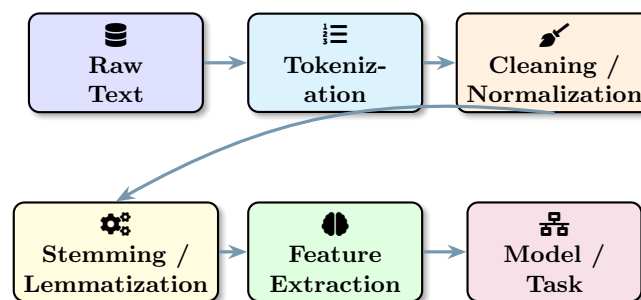


- **Phonology:** Sound patterns of language (relevant to speech recognition and synthesis).
- **Morphology:** Word structure and formation (prefixes, suffixes, roots).
- **Syntax:** Grammatical sentence structure.
- **Semantics:** Literal meaning of words and sentences.
- **Pragmatics:** Meaning in context (speaker intent, implication).
- **Discourse:** Meaning that spans multiple connected sentences.

2 The NLP Pipeline and Text Preprocessing

Text Preprocessing

Text preprocessing converts raw, messy text into a clean, structured form that NLP algorithms can work with, typically the very first stage of any NLP system.



2.1 Core Preprocessing Steps

- **Tokenization:** Splitting text into smaller units — words, sub-words, or sentences.
- **Lowercasing:** Converting all text to lowercase so “Data” and “data” are treated identically.
- **Stopword Removal:** Removing very common, low-information words (“the”, “is”, “and”).
- **Noise Removal:** Stripping punctuation, HTML tags, special characters, and extra whitespace.
- **Stemming:** Crudely chopping word endings using fixed rules (e.g., Porter Stemmer: “studies” → “studi”).
- **Lemmatization:** Using vocabulary and grammar knowledge to return the correct dictionary root, or **lemma** (e.g., “studies” → “study”, “better” → “good”).

Aspect	Stemming	Lemmatization
Approach	Rule-based suffix stripping	Dictionary and morphological analysis
Output	May not be a real word	Always a valid dictionary word
Speed	Faster, lightweight	Slower, needs vocabulary lookup
Accuracy	Lower, cruder	Higher, more linguistically correct

Illustrative Example: Preprocessing a Sentence

Raw text: “The Students are STUDYING Data Science!!!”

- Tokenize: [The, Students, are, STUDYING, Data, Science, !, !, !]
- Lowercase + remove punctuation: [the, students, are, studying, data, science]
- Remove stopwords: [students, studying, data, science]
- Lemmatize: [student, study, data, science]

3 Morphological and Lexical Analysis

Morphology

Morphology is the study of how words are formed from smaller meaningful units called **morphemes**, the smallest grammatical unit of a language.

3.1 Types of Morphemes

- **Free Morpheme:** Can stand alone as a word (e.g., “book”, “run”).
- **Bound Morpheme:** Cannot stand alone; must attach to another morpheme (e.g., “-ing”, “un-”, “-s”).
- **Prefix / Suffix:** Bound morphemes attached before (“un-happy”) or after (“happy-ness”) a root word.

3.2 Lexical Analysis and Vocabulary

- **Lexicon:** The vocabulary of words (and their properties) known to an NLP system.
- **Out-of-Vocabulary (OOV) Words:** Words not seen during training, handled using sub-word tokenization (e.g., Byte-Pair Encoding) in modern systems.
- **Lexical Category:** The part of speech a word belongs to (noun, verb, adjective, etc.).

4 Regular Expressions and Finite-State Methods

Regular Expression

A regular expression (regex) is a compact pattern that describes a set of strings, widely used in NLP for tokenization, cleaning, and simple information extraction.

4.1 Common Regex Building Blocks

Pattern	Meaning	Example Match
[a-z]+	One or more lowercase letters	“hello”
\d+	One or more digits	“2026”
^The	Line starts with “The”	“The dog ran”
cat dog	Either “cat” or “dog”	“dog”

Illustrative Example: Extracting Years with Regex

The pattern `\d{4}` matches any 4-digit number. Applied to “The event was held in 2023 and repeated in 2026”, it extracts 2023 and 2026 — a simple, rule-based way to pull structured information from free text.

4.2 Finite-State Automata (FSA) for Morphology

💡 FSA

A Finite-State Automaton is a simple machine with a finite set of **states** and **transitions** triggered by input symbols. FSAs can compactly recognize valid word forms, e.g., accepting “walk”, “walks”, “walked”, and “walking” but rejecting “walkes”.

- Widely used for tokenization, morphological analysis, and spell-checking before statistical methods became dominant.
- Fast and predictable, but must be manually designed — they do not learn from data like statistical or neural models.

5 Syntax and Parsing

📖 Syntax

Syntax is the set of rules that govern how words combine to form grammatically correct sentences.

5.1 Part-of-Speech (POS) Tagging

💡 POS Tagging

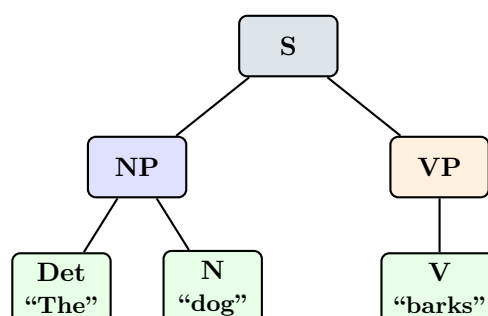
POS tagging assigns a grammatical category (noun, verb, adjective, determiner, etc.) to each word in a sentence, forming the basis for deeper syntactic and semantic analysis.

Example: “The/**DT** dog/**NN** barks/**VB** loudly/**RB**”

5.2 Grammar and Parse Trees

📖 Context-Free Grammar (CFG)

A CFG defines valid sentence structures using production rules, e.g., $S \rightarrow NP VP$, $NP \rightarrow Det N$, $VP \rightarrow V$. Parsing uses these rules to build a **parse tree** showing a sentence’s grammatical structure.



5.3 Constituency vs. Dependency Parsing

- **Constituency Parsing:** Breaks a sentence into nested sub-phrases (as in the tree above), showing hierarchical grouping.
- **Dependency Parsing:** Shows grammatical relationships as directed links between individual words (e.g., “barks” is the root; “dog” is its subject), without grouping into phrases.

6 N-gram Language Models

Language Model

A language model assigns a probability $P(w_1, w_2, \dots, w_n)$ to a sequence of words, capturing how likely that sequence is to occur in natural language.

6.1 The Chain Rule and the Markov Assumption

By the chain rule of probability:

$$P(w_1, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_1, \dots, w_{i-1})$$

Computing this exactly needs impossibly large amounts of data, so n-gram models make the **Markov assumption**: a word depends only on the previous $n - 1$ words. For a **bigram** model ($n = 2$):

$$P(w_1, \dots, w_n) \approx \prod_{i=1}^n P(w_i \mid w_{i-1}), \quad P(w_i \mid w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

6.2 Smoothing

Why Smoothing Is Needed

If a bigram never appeared in training data, its raw probability is zero — which would make the whole sentence probability zero. **Smoothing** reserves a small amount of probability mass for unseen events.

Laplace (add-one) smoothing:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i) + 1}{\text{Count}(w_{i-1}) + V}$$

where V is the vocabulary size.

Beyond Laplace Smoothing

More advanced techniques such as **Good-Turing smoothing** (re-estimates probabilities using the count of counts) and **Kneser-Ney smoothing** (accounts for how many different contexts a word appears in) generally perform better than simple add-one smoothing in real language models.

6.3 Perplexity

☰ Perplexity

Perplexity measures how well a language model predicts a test sequence W of N words — **lower perplexity means a better model**:

$$PP(W) = P(w_1, \dots, w_N)^{-\frac{1}{N}}$$

7 Text Representation Techniques

☰ Text Representation

Since machine learning models require numbers, text must be converted into numerical vectors before it can be processed — this is called **text representation** or **feature extraction**.

7.1 One-Hot Encoding and Bag-of-Words

- **One-Hot Encoding**: Represents each word as a vector of zeros with a single 1 at that word's position in the vocabulary. Vectors are large and carry no notion of similarity.
- **Bag-of-Words (BoW)**: Represents a document as a vector of word counts, ignoring word order and grammar.

7.2 TF-IDF

☰ TF-IDF

Term Frequency–Inverse Document Frequency weighs a term by how often it appears in a document, offset by how common it is across all documents — so common words get lower weight and distinctive words get higher weight.

$$TF(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total terms in } d}, \quad IDF(t) = \log \frac{N}{df_t}, \quad TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

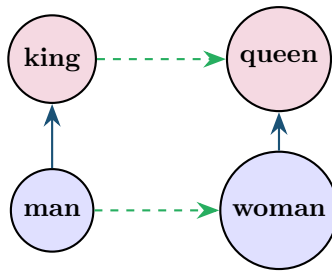
where N is the total number of documents and df_t is the number of documents containing term t .

7.3 Word Embeddings

💡 Word Embeddings

Word embeddings represent each word as a dense, low-dimensional vector learned from large text corpora, such that words with similar meanings have similar vectors (e.g., Word2Vec, GloVe).

- **CBOW (Continuous Bag of Words)**: Predicts a target word from its surrounding context words.
- **Skip-gram**: Predicts the surrounding context words from a single target word.
- **Analogy Property**: Embeddings can capture relationships, famously $\text{king} - \text{man} + \text{woman} \approx \text{queen}$.



7.4 Cosine Similarity

Cosine Similarity

Cosine similarity measures how similar two vectors are by the cosine of the angle between them, regardless of their magnitude — widely used to compare word or document vectors:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

A value near 1 means very similar; a value near 0 means unrelated.

8 Information Retrieval Basics

Information Retrieval

Information Retrieval (IR) is the task of finding the most relevant documents from a large collection in response to a user's query — the core problem solved by search engines.

8.1 The Vector Space Model for Search

Vector Space Model

Both documents and the query are represented as TF-IDF vectors in the same space. Documents are ranked by their **cosine similarity** to the query vector — the higher the similarity, the more relevant the document is judged to be.

8.2 Inverted Index

Inverted Index

An inverted index maps each vocabulary term to the list of documents where it appears, letting a search engine instantly find candidate documents for a query instead of scanning the entire collection.

Term	Posting List (Document IDs)
data	D1, D3, D5
science	D1, D2, D5
model	D2, D3

8.3 Precision and Recall in Retrieval

- **Precision:** Of the documents retrieved, how many are actually relevant.

- **Recall:** Of all relevant documents in the collection, how many were retrieved.
- Search engines typically emphasize precision at the top ranks (e.g., **Precision@10**), since users rarely look past the first page of results.

9 Semantic Analysis

Semantic Analysis

Semantic analysis extracts the actual **meaning** conveyed by words and sentences, going beyond their surface grammatical structure.

9.1 Word Sense Disambiguation (WSD)

WSD

WSD determines which meaning of an ambiguous word is intended in a given context, e.g., “bank” means a financial institution in “I deposited money at the bank” but a riverside in “We sat by the bank.”

9.2 Named Entity Recognition (NER)

NER

NER locates and classifies named entities in text into predefined categories such as **Person**, **Organization**, **Location**, and **Date**.

Example: “[Ali]_{Person} joined [Google]_{Organization} in [Lahore]_{Location} in [2023]_{Date}.”

9.3 Semantic Role Labeling (SRL)

- Identifies “who did what to whom, when, and where” in a sentence by labeling each phrase’s semantic role (Agent, Action, Patient, Location, Time).
- *Example:* “[Ali]_{Agent} [gave]_{Action} [the book]_{Patient} [to Sara]_{Recipient}.”

9.4 Coreference Resolution

Coreference Resolution

Coreference resolution identifies all expressions in a text that refer to the same real-world entity, linking pronouns and noun phrases together.

Example: “Sara finished her homework because she wanted to relax.” — both “her” and “she” corefer with “Sara”.

10 Sequence Models for NLP

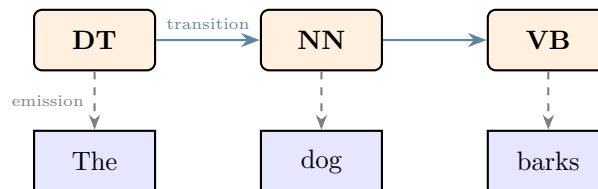
Why Sequence Models?

Language is inherently sequential — the meaning of a word often depends on the words before (and after) it. Sequence models are designed to capture these order-dependent patterns.

10.1 Hidden Markov Models (HMMs) for POS Tagging

💡 HMM for Tagging

An HMM treats POS tags as hidden states and words as observations. It uses **transition probabilities** $P(\text{tag}_i | \text{tag}_{i-1})$ and **emission probabilities** $P(\text{word}_i | \text{tag}_i)$ to find the most likely tag sequence for a sentence (via the Viterbi algorithm).



10.2 RNNs, LSTMs, and GRUs

- **Recurrent Neural Network (RNN)**: Processes a sequence one token at a time, passing a hidden state forward to remember earlier context.
- **Vanishing Gradient Problem**: Plain RNNs struggle to retain information over long sequences during training.
- **LSTM (Long Short-Term Memory)**: Adds input, forget, and output gates to control what information is kept or discarded, handling long-range dependencies far better.
- **GRU (Gated Recurrent Unit)**: A simplified alternative to LSTM with fewer gates, often just as effective and faster to train.

10.3 Sequence-to-Sequence (Seq2Seq) Models

💡 Seq2Seq

A Seq2Seq model uses an **encoder** to compress an input sequence into a context representation, and a **decoder** to generate an output sequence from it — the foundation of neural machine translation and abstractive summarization.

11 Attention and Transformer Architecture

📖 The Problem with RNNs

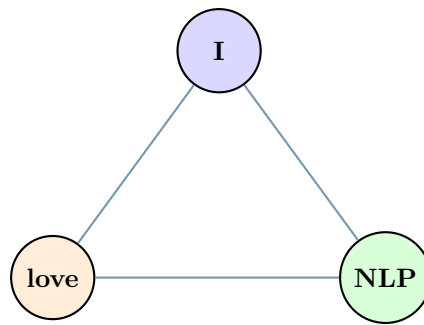
RNNs process words one at a time and can “forget” distant context, and they are slow to train because they cannot be fully parallelized. The **Transformer** architecture (2017) solved both problems using **attention**.

11.1 Self-Attention Mechanism

💡 Self-Attention

Self-attention lets every word in a sentence directly look at every other word and learn how much to “attend to” each one, regardless of distance, using Query (Q), Key (K), and Value (V) vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$



Illustrative Example: Why Attention Helps

“The animal didn’t cross the street because *it* was tired.” Self-attention allows the model to strongly connect *it* with *animal* (not *street*), correctly resolving the reference using context from anywhere in the sentence.

11.2 Transformer Encoder-Decoder



11.3 BERT, GPT, and Large Language Models

Aspect	BERT	GPT
Architecture	Encoder-only	Decoder-only
Training Objective	Masked Language Modeling	Next-token (autoregressive) prediction
Context Direction	Bidirectional	Left-to-right (causal)
Typical Use	Classification, NER, Q&A (understanding)	Text generation, chat (generation)

Lightbulb Large Language Models

LLMs (e.g., GPT-4) are Transformer-based models with billions of parameters, trained on massive text corpora, capable of few-shot learning: performing new tasks from just a few examples in the prompt, without retraining.

12 Core NLP Tasks and Applications

12.1 Common NLP Tasks

- **Text Classification:** Assigning a category to a document (e.g., spam vs. not spam).
- **Sentiment Analysis:** Determining whether text expresses positive, negative, or neutral opinion.
- **Machine Translation:** Automatically translating text from one language to another.
- **Text Summarization:** *Extractive* (selecting key sentences) or *Abstractive* (generating new, condensed sentences).

- **Question Answering:** Automatically producing an answer to a natural-language question, often from a given passage.
- **Chatbots and Dialogue Systems:** Maintaining multi-turn conversations with users.
- **Topic Modeling:** Discovering abstract topics in a collection of documents (e.g., Latent Dirichlet Allocation, LDA).

12.2 Applications of NLP

Domain	Example Applications
Healthcare	Clinical note analysis, symptom extraction, medical Q&A.
Business & IT	Customer sentiment analysis, support chatbots, email spam filters.
Search Engines	Query understanding, semantic search, result ranking.
Translation Services	Real-time machine translation (e.g., Google Translate).
Legal	Contract review, e-discovery, case-law search.
Education	Automated essay scoring, plagiarism detection, tutoring chatbots.
Social Media	Content moderation, trend and misinformation detection.

13 Speech Processing Basics

Speech Processing

Speech processing connects NLP with spoken language, converting between audio signals and text so that voice assistants and dictation tools can understand and respond to spoken input.

13.1 Automatic Speech Recognition (ASR)

ASR

ASR (speech-to-text) converts an audio waveform into text, typically using an acoustic model (mapping sound to phonemes) combined with a language model (choosing the most likely word sequence).

13.2 Text-to-Speech (TTS)

TTS

TTS (speech synthesis) converts written text into natural-sounding spoken audio, used in voice assistants, screen readers, and accessibility tools.

- **Word Error Rate (WER):** The standard metric for ASR accuracy, measuring insertions, deletions, and substitutions relative to a reference transcript.
- **Applications:** Voice assistants (Siri, Alexa), automated transcription, accessibility tools, and voice-controlled IoT devices.

14 Evaluation Metrics in NLP

Why Evaluate?

Different NLP tasks need different evaluation metrics — classification accuracy is meaningless for translation quality, and vice versa.

14.1 Classification Metrics (Recap)

For tasks like spam detection or sentiment classification:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = 2 \cdot \frac{P \cdot R}{P + R}$$

14.2 BLEU Score (Machine Translation)

BLEU

BLEU (Bilingual Evaluation Understudy) compares a machine-generated translation to one or more human reference translations by measuring **n-gram precision** (how many candidate n-grams appear in the reference), combined with a **brevity penalty** that discourages overly short outputs.

14.3 ROUGE Score (Summarization)

ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) measures how much n-gram content from the human reference summary is **recalled** by the machine-generated summary — the complementary counterpart to BLEU's precision focus.

14.4 Perplexity (Recap)

Lower perplexity $PP(W) = P(w_1, \dots, w_N)^{-1/N}$ indicates a language model assigns higher probability to real text, i.e., it models the language better.

15 Ethical and Responsible NLP

Key Concerns

As NLP systems (especially LLMs) are deployed widely, BSIT professionals must understand their risks and responsibilities.

- **Bias:** Models can learn and amplify gender, racial, or cultural biases present in training text (e.g., biased word embeddings or stereotyped generations).
- **Hallucination / Misinformation:** LLMs can generate fluent but factually incorrect or fabricated content with high confidence.
- **Privacy:** Models trained on large text corpora may memorize and leak sensitive personal information.
- **Toxic Content Generation:** Language models can be prompted to produce harmful, offensive, or abusive text if not properly safeguarded.

- **Copyright and Data Provenance:** Training on copyrighted text raises unresolved legal and ethical questions.
- **Job Impact:** Automation of translation, transcription, and customer support may displace certain roles, requiring reskilling.

💡 Responsible NLP Principles

Fairness • Reliability and Safety • Privacy and Security • Inclusiveness • Transparency • Accountability.

16 Core Mathematical Foundations for NLP

16.1 Probability Recap

- **Conditional Probability:** $P(A | B) = \frac{P(A \cap B)}{P(B)}$, the basis of n-gram and HMM models.
- **Bayes' Theorem:** $P(A | B) = \frac{P(B | A)P(A)}{P(B)}$, the foundation of Naive Bayes text classification.
- **Chain Rule:** $P(w_1, \dots, w_n) = \prod_{i=1}^n P(w_i | w_1, \dots, w_{i-1})$, the basis of all language models.

16.2 Why Use Log Probabilities?

💡 Avoiding Numerical Underflow

Multiplying many small probabilities together (as in long sentences) quickly underflows to zero on a computer. NLP systems instead sum **log probabilities**:

$$\log P(w_1, \dots, w_n) = \sum_{i=1}^n \log P(w_i | w_{1:i-1})$$

16.3 Vectors and Similarity Recap

Text representation, introduced earlier, converts words/documents into vectors $\mathbf{A}, \mathbf{B}$; similarity is then computed using the dot product $\mathbf{A} \cdot \mathbf{B} = \sum_i A_i B_i$ and cosine similarity, which underlie nearly all modern retrieval, clustering, and recommendation over text.

17 Exam-Oriented Worked Examples

🔗 Worked Example 1: Bigram Probability from a Mini Corpus

Training corpus: “I love NLP”, “I love AI”, “I love data”.

Counts: $\text{Count}(\text{I}) = 3$, $\text{Count}(\text{I, love}) = 3$, $\text{Count}(\text{love}) = 3$, $\text{Count}(\text{love, NLP}) = 1$.

$$P(\text{love} | \text{I}) = \frac{\text{Count}(\text{I, love})}{\text{Count}(\text{I})} = \frac{3}{3} = 1.0$$

$$P(\text{NLP} | \text{love}) = \frac{\text{Count}(\text{love, NLP})}{\text{Count}(\text{love})} = \frac{1}{3} \approx 0.33$$

</> Worked Example 2: TF-IDF Computation

A document has 50 total words; the term “data” appears 5 times, so $TF = 5/50 = 0.1$. The corpus has $N = 100$ documents, and “data” appears in $df = 10$ of them:

$$IDF = \log_{10} \frac{100}{10} = \log_{10}(10) = 1$$

$$TF\text{-}IDF = 0.1 \times 1 = 0.1$$

</> Worked Example 3: Cosine Similarity

Given vectors $\mathbf{A} = (1, 2, 3)$ and $\mathbf{B} = (2, 3, 1)$:

$$\mathbf{A} \cdot \mathbf{B} = (1)(2) + (2)(3) + (3)(1) = 2 + 6 + 3 = 11$$

$$\|\mathbf{A}\| = \|\mathbf{B}\| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14}$$

$$\cos(\theta) = \frac{11}{\sqrt{14} \times \sqrt{14}} = \frac{11}{14} \approx 0.786$$

The vectors are fairly similar (close to 1).

</> Worked Example 4: Edit (Levenshtein) Distance

Transform “kitten” into “sitting”:

- **kitten** → **sitten** (substitute k → s)
- **sitten** → **sittin** (substitute e → i)
- **sittin** → **sitting** (insert g)

Total edit distance = 3.

</> Worked Example 5: Naive Bayes Text Classification

Classify an email containing the word “offer” as spam or ham, given $P(\text{spam}) = P(\text{ham}) = 0.5$, $P(\text{offer} \mid \text{spam}) = 0.6$, $P(\text{offer} \mid \text{ham}) = 0.1$:

$$P(\text{spam} \mid \text{offer}) \propto 0.5 \times 0.6 = 0.30$$

$$P(\text{ham} \mid \text{offer}) \propto 0.5 \times 0.1 = 0.05$$

Since $0.30 > 0.05$, classify the email as **spam** (normalized confidence $\approx 85.7\%$).

</> Worked Example 6: Perplexity of a Language Model

A bigram model assigns a test sentence of $N = 5$ words a total probability $P = 0.001 = 10^{-3}$:

$$PP(W) = P^{-\frac{1}{N}} = (10^{-3})^{-\frac{1}{5}} = 10^{0.6} \approx 3.98$$

A lower perplexity (closer to 1) would indicate an even better-fitting model.

</> Worked Example 7: Ranking Documents by Cosine Similarity

Query vector $\mathbf{Q} = (1, 1, 0)$; document vectors $\mathbf{D}_1 = (2, 0, 1)$ and $\mathbf{D}_2 = (1, 1, 1)$:

$$\cos(Q, D_1) = \frac{(1)(2) + (1)(0) + (0)(1)}{\sqrt{2}\sqrt{5}} = \frac{2}{\sqrt{10}} \approx 0.632$$

$$\cos(Q, D_2) = \frac{(1)(1) + (1)(1) + (0)(1)}{\sqrt{2}\sqrt{3}} = \frac{2}{\sqrt{6}} \approx 0.816$$

Since $0.816 > 0.632$, the search engine ranks D_2 above D_1 for this query.

⌘ Worked Example 8: Regex Pattern Matching

Text: “Call 555-1234 or 555-5678 for support.” Pattern: `\d{3}-\d{4}`.

The pattern matches two substrings, 555-1234 and 555-5678 — showing how regex extracts structured patterns (like phone numbers) from unstructured text without training any model.

18 Quick Revision Glossary

Term	One-Line Meaning
Tokenization	Splitting text into words, sub-words, or sentences.
Stemming	Rule-based chopping of word endings to an approximate root.
Lemmatization	Dictionary/grammar-based reduction of a word to its true root (lemma).
Stopwords	Very common, low-information words often removed before analysis.
POS Tagging	Assigning a grammatical category to each word in a sentence.
Parsing	Analyzing a sentence’s grammatical structure (constituency or dependency).
Corpus	A large, structured collection of text used to train/evaluate NLP models.
N-gram	A contiguous sequence of n words used to model local context.
Markov Assumption	A word depends only on a limited number of preceding words.
Smoothing	Reserving probability mass for unseen n-grams (e.g., Laplace smoothing).
Perplexity	Measures how well a language model predicts test text; lower is better.
Bag-of-Words	Document representation as word counts, ignoring order.
TF-IDF	Term weighting that balances term frequency against document frequency.
Word Embedding	Dense vector representation of a word capturing semantic meaning.
Word2Vec	Neural method (CBOW/Skip-gram) for learning word embeddings.
Cosine Similarity	Similarity between two vectors based on the angle between them.
NER	Identifying and classifying named entities (person, org, location, date).

Term	One-Line Meaning
WSD	Determining which meaning of an ambiguous word applies in context.
Semantic Role Labeling	Labeling “who did what to whom, when, and where” in a sentence.
HMM	Probabilistic model using hidden states (tags) and observations (words).
Self-Attention	Mechanism letting each word weigh the relevance of every other word.
Transformer	Attention-based architecture underlying BERT, GPT, and modern LLMs.
BERT	Encoder-only Transformer trained via masked language modeling.
GPT	Decoder-only Transformer trained to predict the next token.
BLEU Score	Precision-based metric for evaluating machine translation quality.
ROUGE Score	Recall-based metric for evaluating automatic text summarization.
Regular Expression	A pattern that compactly describes a set of matching strings, used for extraction.
Finite-State Automaton (FSA)	A simple machine with states and transitions, used for rule-based morphology.
Inverted Index	A mapping from each term to the documents containing it, enabling fast search.
Vector Space Model	Represents documents/queries as vectors, ranked by cosine similarity for retrieval.
Coreference Resolution	Linking pronouns and noun phrases that refer to the same real-world entity.
ASR	Automatic Speech Recognition; converts spoken audio into text.
TTS	Text-to-Speech; converts written text into spoken audio.

19 Question Bank Blueprint (80 Marks Support)

💡 Suggested 80 Marks Distribution

- **Section A:** MCQs and definitions (20 marks)
- **Section B:** Short conceptual questions (20 marks)
- **Section C:** Descriptive/theory questions (20 marks)
- **Section D:** Numerical/problem-solving questions (20 marks)

19.1 Important Theory Questions

- Explain the different levels of language analysis with examples.
- Differentiate stemming and lemmatization with examples.
- Describe the complete NLP preprocessing pipeline using a sample sentence.
- Explain how regular expressions and finite-state automata are used in NLP.
- Differentiate constituency parsing and dependency parsing.
- Explain the Markov assumption used in n-gram language models.
- Explain why smoothing is necessary in language modeling.
- Differentiate Bag-of-Words, TF-IDF, and word embeddings as text representations.
- Explain Word2Vec, contrasting the CBOW and Skip-gram approaches.
- Explain the vector space model and how an inverted index speeds up search.
- Differentiate Named Entity Recognition, Word Sense Disambiguation, and Coreference Resolution.
- Explain the self-attention mechanism and why Transformers replaced RNNs.
- Differentiate BERT and GPT in terms of architecture and typical use.
- Differentiate Automatic Speech Recognition (ASR) and Text-to-Speech (TTS).
- Discuss ethical issues in NLP such as bias, hallucination, and privacy.

19.2 Important Numerical/Analytical Questions

- Compute a bigram probability given word and bigram counts from a small corpus.
- Compute TF-IDF for a term given its frequency and corpus statistics.
- Compute cosine similarity between two given word/document vectors.
- Rank two documents against a query using cosine similarity.
- Compute the edit (Levenshtein) distance between two short strings.
- Given a short text and a regex pattern, identify all matching substrings.
- Classify a test document using Naive Bayes given simple probability tables.
- Compute the perplexity of a language model given a test sequence probability.

19.3 Sample MCQs with Answer Key

1. Which process can map “better” to its lemma “good” using vocabulary and grammar knowledge?
(a) Stemming (b) Lemmatization (c) Tokenization (d) Parsing
2. Which formula represents Term Frequency–Inverse Document Frequency weighting?
(a) $TF \times IDF$ (b) $TF + IDF$ (c) TF/IDF (d) $IDF - TF$

3. In the Transformer architecture, which mechanism lets the model relate words regardless of distance?
(a) Convolution (b) Pooling (c) Self-Attention (d) Backpropagation
4. Which metric is most commonly used to evaluate machine translation quality?
(a) Accuracy (b) BLEU (c) Perplexity (d) Entropy
5. BERT is best described as a:
(a) Decoder-only autoregressive model (b) Encoder-only bidirectional model (c) Rule-based grammar checker (d) Purely statistical n-gram model
6. Which NLP task involves identifying names of people, organizations, and locations in text?
(a) POS Tagging (b) Named Entity Recognition (c) Text Summarization (d) Stemming
7. Lower perplexity on a test set indicates:
(a) A worse language model (b) A better language model (c) No relationship to model quality (d) Overfitting only
8. Which data structure allows a search engine to instantly find documents containing a query term?
(a) Parse tree (b) Inverted index (c) Confusion matrix (d) Embedding matrix
9. Coreference resolution is primarily concerned with:
(a) Assigning POS tags (b) Linking expressions that refer to the same entity (c) Measuring translation quality (d) Compressing word vectors

💡 Answer Key

1–(b), 2–(a), 3–(c), 4–(b), 5–(b), 6–(b), 7–(b), 8–(b), 9–(b)

20 Summary

🔧 Key Takeaways

- NLP enables machines to process human language across phonological, morphological, syntactic, semantic, pragmatic, and discourse levels.
- Text preprocessing, regular expressions, and finite-state methods provide the rule-based foundation of every NLP pipeline.
- N-gram language models use the Markov assumption, smoothing, and perplexity to model word sequences probabilistically.
- Text representation has evolved from Bag-of-Words and TF-IDF to dense word embeddings and contextual Transformer representations, underpinning information retrieval via the vector space model.
- Semantic analysis (WSD, NER, SRL, coreference resolution) and sequence models (HMM, RNN, LSTM) extract meaning and structure from language.
- Attention and Transformers (BERT, GPT, LLMs) are the foundation of modern state-of-the-art NLP, complemented by speech processing (ASR/TTS) for spoken language.
- Task-appropriate evaluation (accuracy/F1, BLEU, ROUGE, perplexity) and responsible, ethical use are essential for real-world NLP systems.